

AutoMapper Reverse Mapping in C# with Examples

In this article, I am going to discuss **AutoMapper Reverse Mapping in C#** with Examples. Please read our previous article where we discussed [How to Map Complex Type to Primitive Type using AutoMapper in C#](#) with Examples. This is one of the most important concepts to understand in AutoMapper and at the end of this article, you will understand how AutoMapper Reverse Mapping Works in C# with Examples.

What is AutoMapper Reverse Mapping in C#?

The Automapper Reverse Mapping is nothing but Two-Way mapping which is also called Bidirectional Mapping. As of now, the mapping we discussed is One-Directional Mapping. That means if we have two types let's say Type A and Type B, then we Map Type A with Type B. But using Automapper Reverse mapping it is also possible to Map Type B with Type A.

Example to Understand AutoMapper Reverse Mapping in C#

Let us understand AutoMapper Reverse Mapping in C# with an example. We are going to use the following three classes for this demo. First, create a class file with the name Customer.cs and then copy and paste the following code into it. This is a very simple class having 4 primitive-type properties.

```
namespace AutoMapperDemo
{
    public class Customer
    {
        public int CustomerID { get; set; }
        public string FullName { get; set; }
        public string Postcode { get; set; }
        public string ContactNo { get; set; }
    }
}
```

Next, create a class file with the name Order.cs and then copy and paste the following code into it. This is also a simple class having three primitive properties and one reference type property.

```
namespace AutoMapperDemo
{
    public class Order
    {
        public int OrderNo { get; set; }
        public int NumberOfItems { get; set; }
        public int TotalAmount { get; set; }
        public Customer Customer { get; set; }
    }
}
```

Finally, create a class file with the name OrderDTO.cs and then copy and paste the following code into it. The following class contains the combined properties of the Order and Customer classes.

```
namespace AutoMapperDemo
```

```

{
    public class OrderDTO
    {
        public int OrderId { get; set; }
        public int NumberOfItems { get; set; }
        public int TotalAmount { get; set; }
        public int CustomerId { get; set; }
        public string Name { get; set; }
        public string Postcode { get; set; }
        public string MobileNo { get; set; }
    }
}

```

How to Implement AutoMapper Reverse Mapping in C#?

To implement the AutoMapper Reverse Mapping in C#, we need to call the **ReverseMap** method at the end of the Mapping. So, create a class file with the name MapperConfig.cs and copy and paste the following code into it. Here, you can see, we are Mapping the Source Order object with the Destination OrderDTO object. Further, we are mapping OrderId with the OrderNo properties using the ForMember method as the property name is different. Again, Customer is a Complex type in the Source Object and hence we are mapping that Complex Type to Primitive types of the Destination Object using the ForMember method. And finally, after all the Mapping we are calling the ReverseMap method which makes the Mapping Bi-Directional. Bi-Directional means, now we can make Order as both Source and Destination object as well as we can also make OrderDTO as both Source and Destination Object as per our requirement.

```

using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg => {
                //Mapping Order with OrderDTO
                cfg.CreateMap<Order, OrderDTO>()
                    //OrderId is different so map them using For Member
                    .ForMember(dest => dest.OrderId, act => act.MapFrom(src =>
src.OrderNo))
                    //Customer is a Complex type, so Map Customer to Simple type
                    .ForMember(dest => dest.Name, act => act.MapFrom(src =>
src.Customer.FullName))
                    .ForMember(dest => dest.Postcode, act => act.MapFrom(src =>
src.Customer.Postcode))
                    .ForMember(dest => dest.MobileNo, act => act.MapFrom(src =>
src.Customer.ContactNo))
                    .ForMember(dest => dest.CustomerId, act => act.MapFrom(src =>
src.Customer.CustomerID))
                    .ReverseMap(); //Making the Mapping Bi-Directional
            });
        }
    }
}

```

```

        //Creating the Mapper Instance
        var mapper = new Mapper(config);
        //returning the Mapper Instance
        return mapper;
    }
}
}

```

Using AutoMapper Reverse Mapping in C#:

Now, let us use the AutoMapper Reverse Mapping in C#. Please modify the Main method of the Program class as follows. The following example code is self-explained, so, please go through the comment lines.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Initialize the Mapper
            var mapper = MapperConfig.InitializeAutomapper();
            //Step2: Create the Order Request
            Order OrderRequest = CreateOrderRequest();
            //Step3: Map the OrderRequest object with OrderDTO Object
            var orderDTOData = mapper.Map<Order, OrderDTO>(OrderRequest);
            //or
            //var orderDTOData = mapper.Map<OrderDTO>(OrderRequest);
            //Step4: Print the OrderDTO Data
            Console.WriteLine("After Mapping - OrderDTO Data");
            Console.WriteLine("OrderId : " + orderDTOData.OrderId);
            Console.WriteLine("NumberOfItems : " + orderDTOData.NumberOfItems);
            Console.WriteLine("TotalAmount : " + orderDTOData.TotalAmount);
            Console.WriteLine("CustomerId : " + orderDTOData.CustomerId);
            Console.WriteLine("Name : " + orderDTOData.Name);
            Console.WriteLine("Postcode : " + orderDTOData.Postcode);
            Console.WriteLine("MobileNo : " + orderDTOData.MobileNo);
            Console.WriteLine();
            //Step5: modify the OrderDTO data
            orderDTOData.OrderId = 10;
            orderDTOData.NumberOfItems = 20;
            orderDTOData.TotalAmount = 2000;
            orderDTOData.CustomerId = 5;
            orderDTOData.Name = "Smith";
            orderDTOData.Postcode = "12345";
            //Step6: AutoMapper Reverse Mapping
            mapper.Map(orderDTOData, OrderRequest);
            //Step7: Print Order Data
            Console.WriteLine("After Reverse Mapping - Order Data");
            Console.WriteLine("OrderNo : " + OrderRequest.OrderNo);
        }
    }
}

```

```

        Console.WriteLine("NumberOfItems : " + OrderRequest.NumberOfItems);
        Console.WriteLine("TotalAmount : " + OrderRequest.TotalAmount);
        Console.WriteLine("CustomerId : " + OrderRequest.Customer.CustomerId);
        Console.WriteLine("FullName : " + OrderRequest.Customer.FullName);
        Console.WriteLine("Postcode : " + OrderRequest.Customer.Postcode);
        Console.WriteLine("ContactNo : " + OrderRequest.Customer.ContactNo);
        Console.ReadLine();
    }
    private static Order CreateOrderRequest()
    {
        return new Order
        {
            OrderNo = 101,
            NumberOfItems = 3,
            TotalAmount = 1000,
            Customer = new Customer()
            {
                CustomerID = 777,
                FullName = "James Smith",
                Postcode = "755019",
                ContactNo = "1234567890"
            },
        };
    }
}

```

When you run the application, it displays the data as expected as shown in the below image.

```

After Mapping - OrderDTO Data
OrderId : 101
NumberOfItems : 3
TotalAmount : 1000
CustomerId : 777
Name : James Smith
Postcode : 755019
MobileNo : 1234567890

After Reverse Mapping - Order Data
OrderNo : 10
NumberOfItems : 20
TotalAmount : 2000
CustomerId : 5
FullName : Smith
Postcode : 12345
ContactNo : 1234567890

```

Modifying Order and OrderDTO Class:

Let us modify the Order and OrderDTO class. As you can see in the below classes, now the Complex Type property Customer is present in the **OrderDTO** class and primitive type properties are present in the **Order** class. So, first, modify the Order class as follows. Here, you can see, we have removed the complex Customer property and added primitive properties to hold the customer data.

```
namespace AutoMapperDemo
{
    public class Order
    {
        public int OrderNo { get; set; }
        public int NumberOfItems { get; set; }
        public int TotalAmount { get; set; }
        public int CustomerId { get; set; }
        public string Name { get; set; }
        public string Postcode { get; set; }
        public string MobileNo { get; set; }
    }
}
```

Next, modify the OrderDTO class as follows. Here, you can see, we have removed the primitive properties and added a complex Customer Property to hold the customer data.

```
namespace AutoMapperDemo
{
    public class OrderDTO
    {
        public int OrderId { get; set; }
        public int NumberOfItems { get; set; }
        public int TotalAmount { get; set; }
        public Customer Customer { get; set; }
    }
}
```

Modifying the Mapper Configuration:

Let's implement the AutoMapper ReverseMap() function and see whether we get the results as expected or not. Modify the InitializeAutomapper method of the MapperConfig class as follows. Here, you can see, after the Mapping, we are calling the ReverseMap method which basically set the Mapping between Order and OrderDTO as Two-Way mapping or you can say Bi-Directional Mapping.

```
using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg => {
                //Mapping Order with OrderDTO
            });
        }
    }
}
```

```

        cfg.CreateMap<Order, OrderDTO>()
            .ForMember(dest => dest.OrderId, act => act.MapFrom(src =>
src.OrderNo))
            .ForMember(dest => dest.Customer, act => act.MapFrom(src =>
Customer())
            {
                CustomerID = src.CustomerId,
                FullName = src.Name,
                Postcode = src.Postcode,
                ContactNo = src.MobileNo
            })
            .ReverseMap(); //This will make the Mapping as Bi-Directional
//Now, we can also Map OrderDTO with Order Object
    });
    //Creating the Mapper Instance
    var mapper = new Mapper(config);
    //returning the Mapper Instance
    return mapper;
}
}
}

```

Next, modify the Main method of the Program class as follows.

```

using System;
namespace AutoMapperDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            //Step1: Initialize the Mapper
            var mapper = MapperConfig.InitializeAutomapper();

            //Step2: Create the Order Request
            var OrderRequest = CreateOrderRequest();

            //Step3: Map the OrderRequest object to Order DTO
            var orderDTOData = mapper.Map<Order, OrderDTO>(OrderRequest);

            //Step4: Print the OrderDTO Data
            Console.WriteLine("After Mapping - OrderDTO Data");
            Console.WriteLine("OrderId : " + orderDTOData.OrderId);
            Console.WriteLine("NumberOfItems : " + orderDTOData.NumberOfItems);
            Console.WriteLine("TotalAmount : " + orderDTOData.TotalAmount);
            Console.WriteLine("CustomerId : " + orderDTOData.Customer.CustomerId);
            Console.WriteLine("FullName : " + orderDTOData.Customer.FullName);
            Console.WriteLine("Postcode : " + orderDTOData.Customer.Postcode);
            Console.WriteLine("ContactNo : " + orderDTOData.Customer.ContactNo);
            Console.WriteLine();
        }
    }
}

```

```

//Step5: modify the OrderDTO data
orderDTOData.OrderId = 10;
orderDTOData.NumberOfItems = 20;
orderDTOData.TotalAmount = 2000;
orderDTOData.Customer.CustomerID = 5;
orderDTOData.Customer.FullName = "Pranaya Rout";
orderDTOData.Customer.Postcode = "12345";
orderDTOData.Customer.ContactNo = "0011220034";
//Step6: Reverse Mapping
mapper.Map(orderDTOData, OrderRequest);
//Step7: Print the Order Data
Console.WriteLine("After Reverse Mapping - Order Data");
Console.WriteLine("OrderNo : " + OrderRequest.OrderNo);
Console.WriteLine("NumberOfItems : " + OrderRequest.NumberOfItems);
Console.WriteLine("TotalAmount : " + OrderRequest.TotalAmount);
Console.WriteLine("\nCustomerId : " + OrderRequest.CustomerId);
Console.WriteLine("Name : " + OrderRequest.Name);
Console.WriteLine("Postcode : " + OrderRequest.Postcode);
Console.WriteLine("MobileNo : " + OrderRequest.MobileNo);
Console.ReadLine();
}
private static Order CreateOrderRequest()
{
    return new Order
    {
        OrderNo = 101,
        NumberOfItems = 3,
        TotalAmount = 1000,
        CustomerId = 777,
        Name = "James Smith",
        Postcode = "755019",
        MobileNo = "1234567890"
    };
}
}
}
}

```

Now run the application and see the output.

```
After Mapping - OrderDTO Data
OrderId : 101
NumberOfItems : 3
TotalAmount : 1000
CustomerId : 777
FullName : James Smith
Postcode : 755019
ContactNo : 1234567890
```

```
After Reverse Mapping - Order Data
OrderNo : 10
NumberOfItems : 20
TotalAmount : 2000
```

```
CustomerId : 777
Name : James Smith
Postcode : 755019
MobileNo : 1234567890
```



Primitive Type Working



Complex Type Not Working

As you can see from the above output, the AutoMapper Reverse Mapping work as expected for the primitive types but not working for the complex type. So when both classes have members that are named equally then AutoMapper ReverseMap() function works as expected. But if the classes contain members that are different and not mapped via the default (per naming) mapping then this does not work as expected.

How do make the two mapping work as expected?

If you want to make the two-way mapping work as expected then you need to do the Mapping via ForMember and need to tell the mapping for the complex type. So, modify the InitializeAutomapper method of the MapperConfig class as follows. Here, you can see, after the ReverseMap method, we are also using the ForMember method to map the Complex Type Property of the OrderDTO Object to the Primitive Type Properties of the Order Object.

```
using AutoMapper;
namespace AutoMapperDemo
{
    public class MapperConfig
    {
        public static Mapper InitializeAutomapper()
        {
            //Configuring AutoMapper
            var config = new MapperConfiguration(cfg => {
                //Mapping Order with OrderDTO
                cfg.CreateMap<Order, OrderDTO>()
                    .ForMember(dest => dest.OrderId, act => act.MapFrom(src =>
src.OrderNo))
```

```

        .ForMember(dest => dest.Customer, act => act.MapFrom(src =>
Customer()
    {
        CustomerID = src.CustomerId,
        FullName = src.Name,
        Postcode = src.Postcode,
        ContactNo = src.MobileNo
    )))
    .ReverseMap() //This will make the Mapping as Bi-Directional
    //Mapping Complex Type to Primitive Type Properties
    .ForMember(dest => dest.CustomerId, act => act.MapFrom(src =>
src.Customer.CustomerID))
    .ForMember(dest => dest.Name, act => act.MapFrom(src =>
src.Customer.FullName))
    .ForMember(dest => dest.MobileNo, act => act.MapFrom(src =>
src.Customer.ContactNo))
    .ForMember(dest => dest.Postcode, act => act.MapFrom(src =>
src.Customer.Postcode));
    });
    //Creating the Mapper Instance
    var mapper = new Mapper(config);
    //returning the Mapper Instance
    return mapper;
}
}
}

```

With the above changes in place, now run the application and you will see the output as expected as shown in the below image.

After Mapping - OrderDTO Data

OrderId : 101
NumberOfItems : 3
TotalAmount : 1000
CustomerId : 777
FullName : James Smith
Postcode : 755019
ContactNo : 1234567890

After Reverse Mapping - Order Data

OrderNo : 10
NumberOfItems : 20
TotalAmount : 2000

CustomerId : 5
Name : Pranaya Rout
Postcode : 12345
MobileNo : 0011220034

In the next article, I am going to discuss [AutoMapper Conditional Mapping in C#](#) with Examples. Here, in this article, I try to explain **AutoMapper Reverse Mapping in C#** with Examples. I hope this AutoMapper ReverseMapping article will help you with your need. I would like to have your feedback. Please post your feedback, question, or comments about this article.